



4 动态规划

Dynamic Programming



回顾：分治策略

施行分治策略是基于以下几点认识：

- 小问题比大问题更容易解决
- 将小问题的解组装成大问题的解，所需的成本低于直接求解大问题的成本
- 小问题又可以按照同样的方式分解为更小的问题



引例：费氏数列

- 费氏数列是由13世纪的意大利数学家、来自Pisa的 Leonado Fibnacci发现。
- 费氏数列是由0，1开始，之后的每一项等于前两项之和：
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144.....。
- 这个数列有如下一些特性：
 - 前2个数相加等于第3个数
 - 前1个数除以后一个数越往后越无限接近于0.618 (黄金分割)
 - 相邻的两个比率必是一个小于0.618一个大于0.618
 - 后1个数除以前一个数越往后越无限接近于1.618
 - ...



$$f(n) = \begin{cases} 1 & , \text{ if } n = 1, 2 \\ f(n-1) + f(n-2) & , \text{ if } n \geq 3 \end{cases}$$

递归形式的算法：

```
procedure Fib(n)
```

```
  if n=1 or n=2 then return 1
```

```
  else return Fib(n-1)+Fib(n-2)
```

优点：



简洁，容易书写以及调试。

缺点：



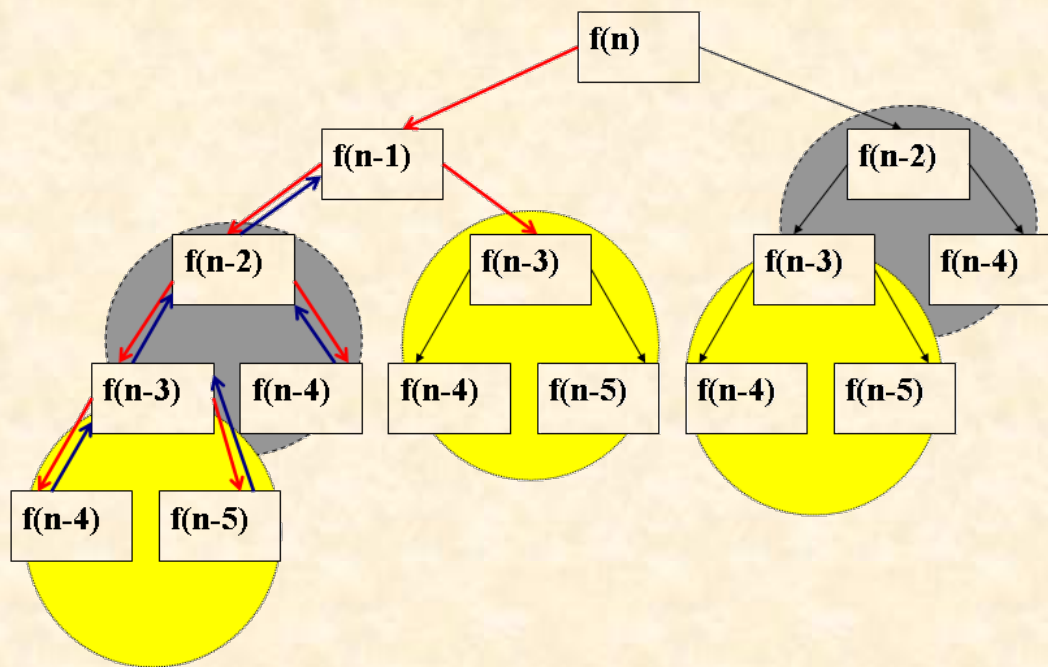
效率低下。



为何效率低下？

- 使用直观的方式分析

存在大量重复计算



- 使用时间复杂性的方式分析

$$T(n) = \begin{cases} 1 & \text{if } n = 1, 2 \\ T(n-1) + T(n-2) & \text{if } n \geq 3 \end{cases}$$


$$T(n) \approx \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^n \approx 0.447(1.618)^n$$

即时间复杂度为输入规模的指数形式。当 $n=100$ 时，用递归求解的时间 $T(100) \approx 3.53 \times 10^{20}$ ，若每秒计算 10^8 次，需111,935年！



解决方法

- 借助于变量存储中间计算结果，**消除重复计算**。代码片断如下：

```
f1 ← 1  
f2 ← 1  
for i ← 3 to n  
    result ← f1+f2  
    f1 ← f2  
    f2 ← result  
end for  
return result
```

$$T(n) = \Theta(n)$$

$$S(n) = \Theta(1)$$



子问题的重复出现

这类问题的特点：大量子问题是重复出现的

递归法求解，将导致大量重复的递归调用，效率低下。

解决途径：动态规划法



与分治法的区别

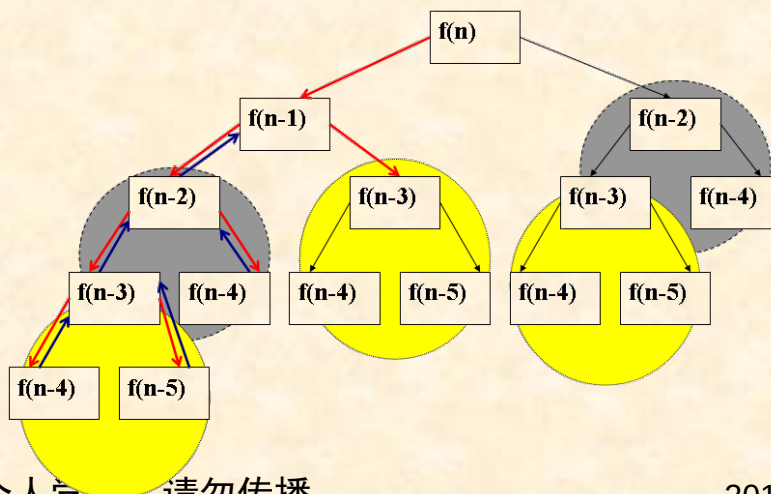
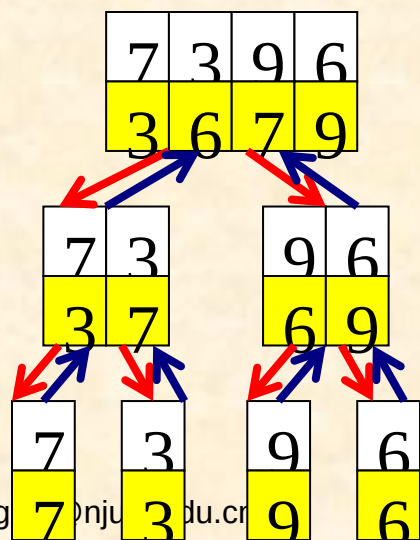
与分治法类似，动态规划法也是

把问题层层分解为规模较小的同类子问题。

二者之间一个重要的不同点在于，

分治法分解后得到的子问题通常都是相互独立的，

而动态规划法分解后得到的子问题很多都是重复的。





例如使用动态规划法求Fibonacci序列的第n项,:

$$f_1=1, f_2=1; \{\text{初值已知}\}$$

$$f_3 = f_2 + f_1 = 1 + 1 = 2$$

$$f_4 = f_3 + f_2 = 2 + 1 = 3$$

...

$$f_n = f_{n-1} + f_{n-2} = \dots$$

时间复杂度: $\Theta(n)$, 空间复杂度: $\Theta(1)$ (常数)

特点: 自底向上, 中间结果保存备用

因此对这类问题(子问题很多都是重复的), 比较明智的做法是: 对重复出现的子问题, 只是在第一次遇到时才进行计算, 然后把计算所得的结果保存起来; 当再次遇到该子问题时, 就直接引用已保存的结果, 而不再重新求解



动态规划的基本思想

- 动态规划的实质是**分治**和**消除冗余**，是一种将问题实例分解为更小的、相似的子问题，并存储子问题的解以避免计算重复的子问题，来解决最优化问题的算法策略。
- 基本步骤：
 - 分析一个最优解应该具备的结构。
 - 递归地定义其最优解。
 - 以自底向上的方式计算出最优值。
 - 根据计算最优值时得到的信息，构造最优解。



矩阵链相乘

- 给定 n 个连乘的矩阵 $M_1 \cdot M_2 \dots M_{n-1} \cdot M_n$ ，问：所需要的最小乘法次数是多少次？对应此最小乘法次数，矩阵是按照什么结合方式相乘的？

$$\begin{array}{lcl} (M_1)_{2 \times 10} \cdot (M_2)_{10 \times 2} \cdot (M_3)_{2 \times 10} & \begin{array}{l} \nearrow \\ \searrow \end{array} & \begin{array}{l} (M_1 \cdot M_2) \cdot M_3 \quad 2 \times 10 \times 2 + 2 \times 2 \times 10 = 80 \\ M_1 \cdot (M_2 \cdot M_3) \quad 10 \times 2 \times 10 + 2 \times 10 \times 10 = 400 \end{array} \end{array}$$

观察发现：多个矩阵连乘时，相乘的结合方式不同，所需要的乘法次数大不相同。

$$(A)_{p \times q} \cdot (B)_{q \times r} \quad \text{所需要的乘法次数为:} \quad p \times q \times r$$



$M_1 \cdot M_2 \cdot M_3 \cdots M_n$ 按照何种结合方式相乘，所需要的乘法次数最少？

穷举法：

1. 找出所有可能的相乘结合方式；
2. 计算每种相乘结合方式所需要的乘法次数；
3. 求min；

$f(n)$ 表示 n 个矩阵连乘所有可能的结合方式, 下面设法求出其解析解。

$$\underbrace{(M_1 \cdot M_2 \cdot M_3 \cdots M_k)}_{f(k)} \cdot \underbrace{(M_{k+1} \cdots M_n)}_{f(n-k)}$$

$$f(n) = \sum_{k=1}^{n-1} f(k) \cdot f(n-k)$$

$$f(1)=1, f(2)=1, f(3)=2 \implies f(n) = \frac{1}{n} C_{2n-2}^{n-1} \xrightarrow{n! \approx \sqrt{2\pi n} (n/2)^n}$$

$$f(n) = \frac{(2n-2)!}{n((n-1)!)^2} \approx \frac{4^n}{4\sqrt{\pi}n^{1.5}} \implies f(n) = \Omega\left(\frac{4^n}{n^{1.5}}\right)$$



使用动态规划法：

$$M_1 \cdot M_2 \cdot M_3 \cdots M_n$$

$$r_1, r_2, r_3, \cdots, r_n, r_{n+1}$$

$$(M_i)_{r_i \times r_{i+1}} \quad 1 \leq i \leq n$$

$$M_{i,j} = M_i \cdot M_{i+1} \cdots M_{j-1} \cdot M_j$$

$C[i, j]$ ：计算 $M_{i,j}$ 所需的最小乘法次数。

$i=1, j=n$ 时，原问题得解。

$$\underbrace{M_{i,j}}_{C[i,j]} = \underbrace{(M_i \cdot M_{i+1} \cdots M_{k-1})}_{C[i,k-1]} \cdot \underbrace{(M_k \cdot M_{k+1} \cdots M_{j-1} \cdot M_j)}_{C[k,j]}$$

$\longleftarrow k \longrightarrow$

$$C[i, j] = C[i, k-1] + C[k, j] + r_i \cdot r_k \cdot r_{j+1}$$



$$C[i, j] = \min_{i < k \leq j} \{C[i, k-1] + C[k, j] + r_i \cdot r_k \cdot r_{j+1}\}$$



一个实例

$$(M_1)_{5 \times 10} \cdot (M_2)_{10 \times 4} \cdot (M_3)_{4 \times 6} \cdot (M_4)_{6 \times 10} \cdot (M_5)_{10 \times 2}$$

$$r_1 = 5, r_2 = 10, r_3 = 4, r_4 = 6, r_5 = 10, r_6 = 2$$

$d = 0$

$d = 1$

$d = 2$

$d = 3$

$d = 4$

$C[1,1]=0$ (M_1)	$C[1,2]=200$ ($M_1 M_2$)	$C[1,3]=320$	$C[1,4]=620$	$C[1,5]=348$
	$C[2,2]=0$ (M_2)	$C[2,3]=240$ ($M_2 M_3$)	$C[2,4]=640$ (M_2) ($M_3 M_4$)	$C[2,5]=248$
		$C[3,3]=0$ (M_3)	$C[3,4]=240$ ($M_3 M_4$)	$C[3,5]=168$
			$C[4,4]=0$ (M_4)	$C[4,5]=120$ ($M_4 M_5$)
				$C[5,5]=0$ (M_5)

$$C[2,4] = \min_{2 \leq k \leq 4} \{C[2, k-1] + C[k, 4] + r_2 \cdot r_k \cdot r_{4+1}\}$$

$$\begin{aligned} k=3 &\rightarrow C[2,4]=C[2,2]+C[3,4]+r_2 \cdot r_3 \cdot r_{4+1} = 0 + 240 + 10 \cdot 4 \cdot 10 = 640 \rightarrow (M_2) \cdot (M_3 \cdot M_4) \\ k=4 &\rightarrow C[2,4]=C[2,3]+C[4,4]+r_2 \cdot r_4 \cdot r_{4+1} = 240 + 0 + 10 \cdot 6 \cdot 10 = 840 \rightarrow (M_2 \cdot M_3) \cdot (M_4) \end{aligned} \} \min$$



输入： $r[1..n+1]$: 表示 n 个矩阵规模的 $n+1$ 个整数.

输出： n 个矩阵连乘的最小乘法次数.

```
1. for  $i \leftarrow 1$  to  $n$  {填充对角线  $d_0$ }
2.    $C[i,i] \leftarrow 0$ 
3. end for
4. for  $d \leftarrow 1$  to  $n-1$  {填充对角线  $d_1$  到  $d_{n-1}$ }
5.   for  $i \leftarrow 1$  to  $n-d$  {填充对角线  $d_i$  的每个项目}
6.      $j \leftarrow i+d$  {该对角线上  $j, i$  满足的关系}
7.      $C[i,j] \leftarrow \infty$ 
8.     for  $k \leftarrow i+1$  to  $j$ 
9.        $C[i,j] \leftarrow \min\{ C[i,j], C[i,k-1] + C[k,j] + r_i \times r_k \times r_{j+1} \}$ 
10.    end for
11.  end for
12. end for
13. return  $C[1,n]$ 
```

$$T(n) = \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} \sum_{\substack{j \\ k=i+1}}^j c = \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} \sum_{\substack{i+d \\ k=i+1}}^{i+d} c = \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} \sum_{k=1}^d c = \Theta(n^3)$$



$$\begin{aligned} T(n) &= \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} \sum_{\substack{j \\ k=i+1}}^j c = \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} \sum_{\substack{i+d \\ k=i+1}}^{i+d} c = \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} \sum_{\substack{d \\ k=1}}^d c \\ &= \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} cd = c \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} d \\ &= c \left(\sum_{i=1}^{n-1} 1 + \sum_{i=1}^{n-2} 2 + \sum_{i=1}^{n-3} 3 + \dots + \sum_{i=1}^{n-(n-1)} (n-1) \right) \\ &= c \left((n-1) \cdot 1 + (n-2) \cdot 2 + (n-3) \cdot 3 + \dots + (n-(n-1)) \cdot (n-1) \right) \\ &= c \left(n \cdot 1 + n \cdot 2 + n \cdot 3 + \dots + n \cdot (n-1) - 1 \cdot 1 - 2 \cdot 2 - (n-1) \cdot (n-1) \right) \\ &= c \left(n(1+2+3+\dots+(n-1)) - \sum_{k=1}^{n-1} k^2 \right) \\ &= c \left(n \cdot \frac{n(n-1)}{2} - \frac{1}{6}(n-1)n(2n-1) \right) \\ &= \frac{1}{6} (cn^3 - cn) = \Theta(n^3) \end{aligned}$$



思考：

从蛮力法指数量级的时间降低到了 $\Theta(n^3)$ ，原因呢？

- 关键在于蛮力法是**将加括号的所有情况进行穷举**比较，
- 而上述方法**只对那些可能产生最优解的情况**进行考虑和比较，从而大大减少了进行比较的时间复杂度。

例如： **$((M_1M_2)M_3)$** **$(M_4(M_5M_6))$** 是理论最优解

蛮力法在求解过程中，仍要考虑 **$((M_1M_2)M_3)$** **$((M_4M_5)M_6)$**

而动态规划法根本不予考虑这种情形。



动态规划算法的两个主要特点：

注意一下上述的计算过程可以看到，求解 $C[i,j]$ 时，规模较小的 $C[i,k]$ 和 $C[k+1,j]$ ($i \leq k < j$) 会被多次用到，这就是子问题的高度重复性。

适宜用动态规划法求解问题时的另一个主要特征是所谓的最优子结构性质：问题的最优解中包含着其每一个子问题的最优解。

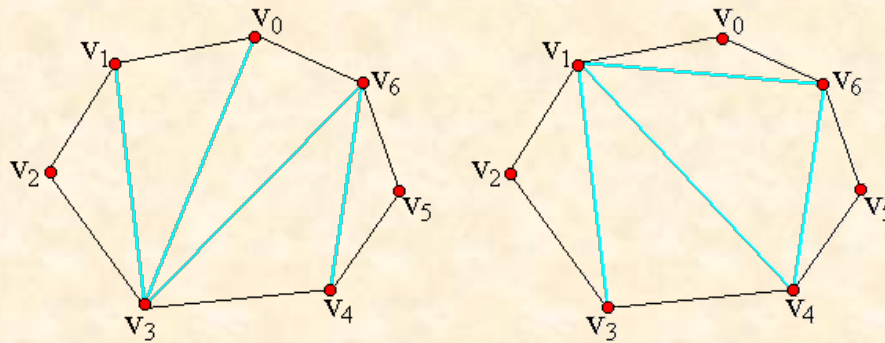


平面凸多边形最优三角划分

- 平面多边形
 - 由在同一平面且不在同一直线上的多条线段首尾顺次连结且不相交所组成的图形叫做多边形。
- 平面凸多边形
 - 弦：连接平面多边形的任意两个不同顶点的线段。
 - 平面凸多边形：如果一个平面多边形的任意一条弦，要么在该多边形的内部，要么恰好为该多边形的边，那么，称该平面多边形为凸的。
- 三角划分
 - 将平面凸多边形分割成互不相交的三角形。



平面凸多边形的表示：用平面凸多边形顶点的逆时针序列表示凸多边形，即 $P=\{v_0, v_1, \dots, v_n\}$ 表示具有 $n+1$ 条边的平面凸多边形。



1. 给定一个平面凸多边形 P ，其三角划分不是唯一的。
2. 给定平面凸多边形 P ，对于 P 的每种三角划分，可以定义一个权函数(例如：三角划分中所有三角形的边长之和)。
3. 最优三角划分：使得权函数取最小值的三角划分。

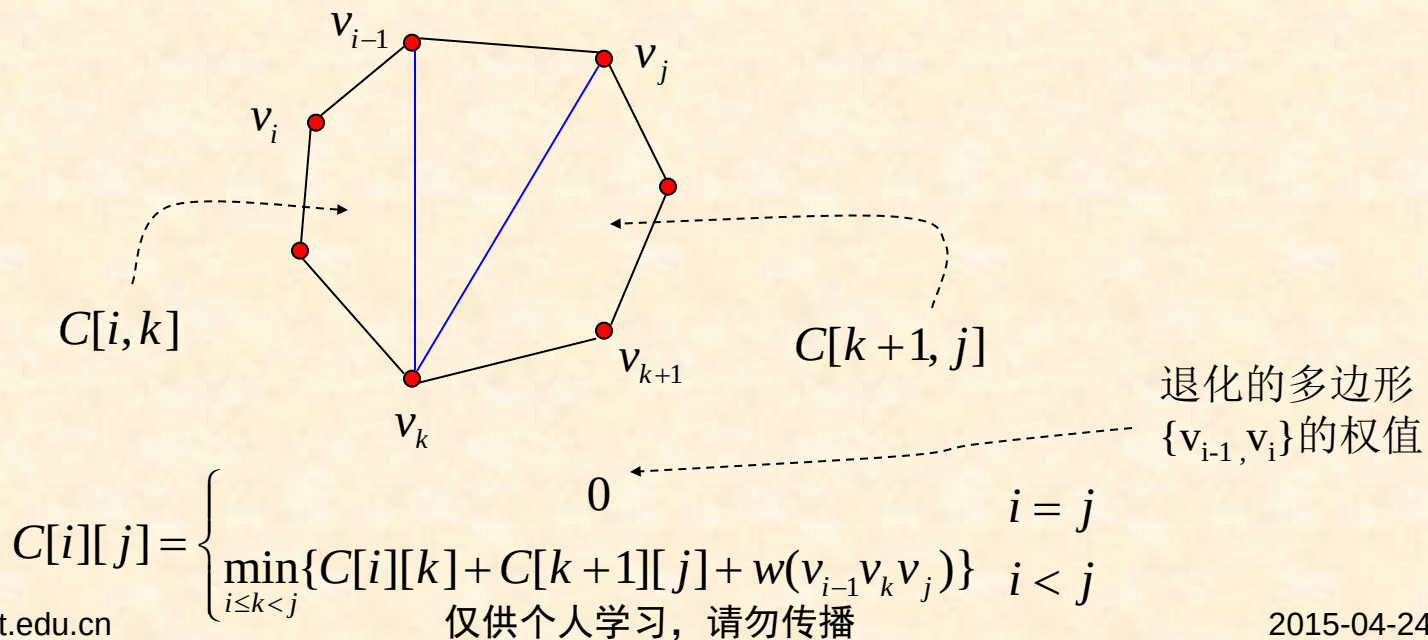


问题：给定平面凸多边形 $P=\{v_0, v_1, \dots, v_n\}$ ，求： P 的最优三角划分。
(不妨假设：权函数定义为三角划分中所有三角形的边长之和)
现在要求：权函数的最小值，以及对应该最小值的三角划分方式。

↓ 类比

矩阵连乘：最小乘法次数，以及对应该最小乘法次数的矩阵结合方式。

若 $P=\{v_0, v_1, \dots, v_n\}$ 是一个凸多边形，那么 $\{v_{i-1}, v_i, \dots, v_j\}$ 所构成的必定也是一个凸多边形。定义 $C[i, j]$ 为子凸多边形 $\{v_{i-1}, v_i, \dots, v_j\}$ 的最优三角划分所对应的权函数值，即其最优值。





最长公共子序列问题

- 给定两个定义在字符集 Σ 上的字符串A和B，长度分别为n和m，现在要求它们的最长公共子序列的长度值(最优值)，以及对应的子序列(最优解)。
- 子序列

$A = a_1 a_2 \cdots a_n$ 的一个子序列是形如下式的一个字符串： $a_{i_1} a_{i_2} \cdots a_{i_k}$ ，其中
 $1 \leq i_1 < i_2 < \cdots < i_k \leq n$

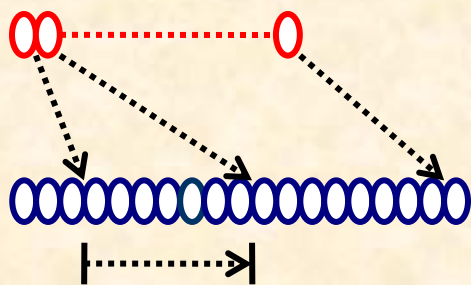
例如： $A = zxy$ 子序列可以是：“”，z, x, y, zx, zy, xy, zxy。

但是xz, yz, xyz不是它的子序列。

$$C_n^0 + C_n^1 + \cdots + C_n^n = 2^n$$



- 穷举法(Brute-Force):
 - 找出A字符串所有可能的子序列(2^n);
 - 对于A的每一个子序列, 判断其是否是B的一个子序列, 需要的时间为 $\Theta(m)$;
 - 求max; 总的时间为 $\Theta(m 2^n)$.





$$A = a_1 a_2 \cdots a_n \quad B = b_1 b_2 \cdots b_m$$

$$\left. \begin{array}{l} a_1 a_2 \cdots a_{i-1} a_i \\ b_1 b_2 \cdots b_{j-1} b_j \end{array} \right\} \xrightarrow{\text{最长公共子序列的长度值}} C[i, j]$$

$$C[i, j] = \begin{cases} 0 & \text{if } i == 0 \text{ or } j == 0 \\ C[i-1, j-1] + 1 & \text{if } i > 0 \text{ \& } j > 0 \text{ \& } a_i == b_j \\ \max\{C[i, j-1], C[i-1, j]\} & \text{if } i > 0 \text{ \& } j > 0 \text{ \& } a_i \neq b_j \end{cases}$$

$$\begin{array}{l} \underline{a_1 a_2 \cdots a_{i-1} a_i} \\ \underline{b_1 b_2 \cdots b_{j-1} b_j} \end{array} \Rightarrow C[i, j-1]$$

$$\begin{array}{l} \underline{a_1 a_2 \cdots a_{i-1} a_i} \\ \underline{b_1 b_2 \cdots b_{j-1} b_j} \end{array} \Rightarrow C[i-1, j]$$

} max

(A dashed arrow points from the 'max' label back to the third case of the recurrence relation above.)



一个实例: $A = xyxxz$ $B = zxzyyz$

使用一个 $(n+1) \times (m+1)$ 的表格C来进行计算。 $C(i, \cdot)$ 表示A(i), $C(\cdot, j)$ 表示B(j)
逐行填满表格, 问题得解。引入 $\nwarrow \uparrow \leftarrow$ 符号表示求解痕迹。

	0	1,z	2,x	3,z	4,y	5,y	6,z
0	0	0	0	0	0	0	0
1,x	0	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\leftarrow$	1 $\leftarrow$	1 $\leftarrow$
2,y	0	0 $\uparrow$	1 $\uparrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$	2 $\leftarrow$
3,x	0	0 $\uparrow$	1 $\uparrow$	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	2 $\uparrow$
4,x	0	0 $\uparrow$	1 $\uparrow$	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	2 $\uparrow$
5,z	0	0 $\uparrow$	1 $\uparrow$	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$

因为 $a_3 \neq b_4$, 所以 $C[3, 4] = \max\{C[3, 3], C[2, 4]\} = \max\{1, 2\} = 2$

因为 $a_5 = b_6$, 所以 $C[5, 6] = C[4, 5] + 1 = 2 + 1 = 3$



输入：两个字符串A, B, 长度分别为n, m.

输出：X和Y的最长公共子序列长度.

```
1.  for i ← 0 to n
2.    C[i,0] ← 0
3.  end for
4.  for j ← 0 to m
5.    C[0,j] ← 0
6.  end for
7.  for i ← 1 to n
8.    for j ← 1 to m
9.      if ai=bj then C[i, j] ← C[i-1, j-1]+1
10.         B[i,j] ← "↖"
10.     else if C[i-1,j] ≥ C[i,j-1]
11.       then C[i, j] ← C[i-1,j]
12.         B[i,j] ← "↑"
13.     else C[i, j] ← C[i,j-1]
14.       B[i,j] ← "←"
15.     end if
16.   end for
17. end for
18. Return C and B
```

$$T(n) = \Theta(nm)$$



例2: $X=\langle abcbabdab \rangle$; $Y=\langle bdcaba \rangle$

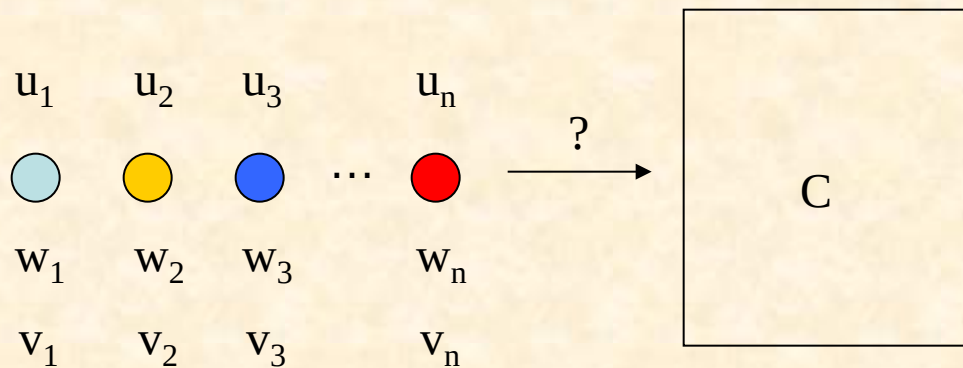
	0	1,b	2,d	3,c	4,a	5,b	6,a
0	0	0	0	0	0	0	0
1,a	0	0 ↑	0 ↑	0 ↑	1 ↖	1 ←	1 ↖
2,b	0	1 ↖	1 ←	1 ←	1 ↑	2 ↖	2 ←
3,c	0	1 ↑	1 ↑	2 ↖	2 ←	2 ↑	2 ↑
4,b	0	1 ↖	1 ↑	2 ↑	2 ↑	3 ↖	3 ←
5,d	0	1 ↑	2 ↖	2 ↑	2 ↑	3 ↑	3 ↑
6,a	0	1 ↑	2 ↑	2 ↑	3 ↖	3 ↑	4 ↖
7,b	0	1 ↖	2 ↑	2 ↑	3 ↑	4 ↖	4 ↑

- 1) 从最大值开始沿箭头方向跟踪，可得到LCS.
- 2) LCS可能不止一个.
- 3) 也可以将↑都改为←，同样可以得到LCS.



0-1背包问题

- 给定 n 个物品 $\{u_1, u_2, \dots, u_n\}$ 和一个背包，物品 i 的重量为 w_i ，价值为 v_i ，已知背包的承重量为 C 。问：在不撑破背包的条件下，选择哪些物品装入背包，得到的总价值最大？
- 之所以称为0-1 背包问题，是因为一个物品要么装入、要么不装入，这两种状态分别用1 和0 表示。





0-1背包问题的形式化描述:

给定 $C>0$, $w_i>0$, $v_i>0$, $1\leq i\leq n$, 找出一个 n 元的0-1 向量 $(x_1, x_2, \dots, x_n)$, $x_i \in \{0, 1\}$, $1\leq i\leq n$, 求如下优化问题:

$$\max \sum_{i=1}^n v_i x_i$$

$$s.t. \quad \sum_{i=1}^n w_i x_i \leq C, \quad x_i \in \{0, 1\}, 1 \leq i \leq n$$



背包问题的最优子结构性质

假设我们已经找到了原问题的最优解 $(y_1, y_2, \dots, y_n)$, 仔细分析可以发现, $(y_1, \dots, y_{n-1})$ 是如下问题的最优解:

$$\begin{aligned} \max \quad & \sum_{i=1}^{n-1} v_i x_i \\ \text{s.t.} \quad & \sum_{i=1}^{n-1} w_i x_i \leq C - w_n y_n \\ & x_i \in \{0, 1\}, 1 \leq i \leq n-1 \end{aligned}$$

证明用反证法。假设 $(y_1, \dots, y_{n-1})$ 不是该问题的最优解, 而

$(z_1, \dots, z_{n-1})$ 是该问题的最优解。则有 $\sum_{i=1}^{n-1} v_i z_i > \sum_{i=1}^{n-1} v_i y_i$, 同时满足

约束条件: $\sum_{i=1}^{n-1} w_i z_i < C - w_n y_n$



则有

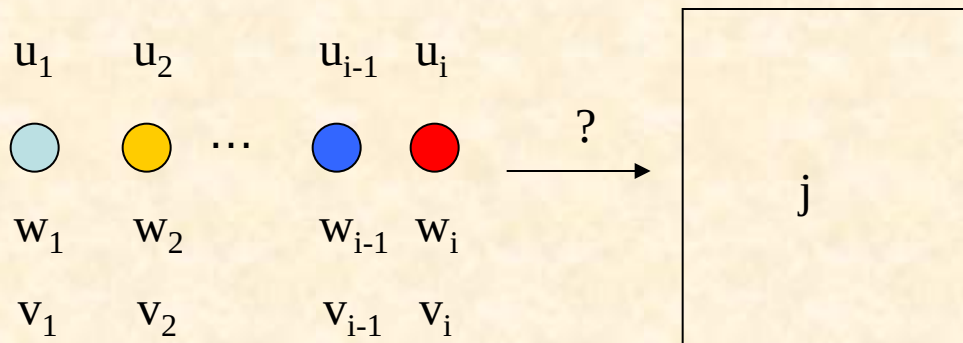
$$\sum_{i=1}^{n-1} v_i \mathbf{z}_i + v_n y_n > \sum_{i=1}^{n-1} v_i \mathbf{y}_i + v_n y_n = \sum_{i=1}^n v_i \mathbf{y}_i$$

这说明：对于原问题，装包方案 $(z_1, z_2, \dots, z_{n-1}, \mathbf{y}_n)$ 比原问题的最优解 $(y_1, y_2, \dots, y_n)$ 更好！这显然与假设矛盾，命题得证。

这个结论说明：该问题具有最优子结构性质。

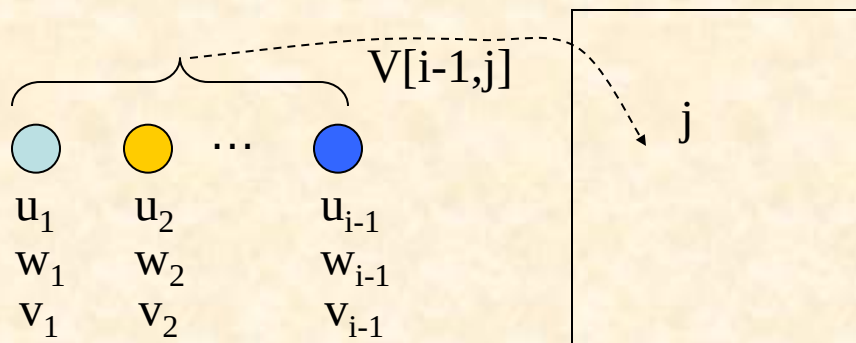


设 $V[i,j]$ 表示从前 i 个物品 $\{u_1, u_2, \dots, u_i\}$ 中取出一部分装入承重量为 j 的背包所能取得的最大价值。那么，当 $i=n, j=C$ 时， $V[n,C]$ 就是原问题的解。

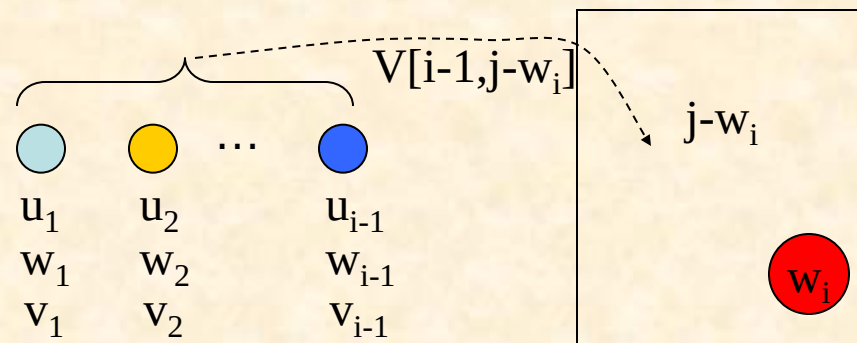


$$V[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ V[i-1, j] & \text{if } j < w_i \\ \max\{V[i-1, j], V[i-1, j-w_i] + v_i\} & \text{if } i > 0 \text{ and } j \geq w_i \end{cases}$$

Case 1:



Case 2:





输入：物品集合 $\{u_1, u_2, \dots, u_n\}$ ，重量分别为 $w_1, w_2, \dots, w_n$ ，价值分别为 $v_1, v_2, \dots, v_n$ ，承重量为 C 的背包

输出：背包所能装物品的最大价值

```
1. for  $i \leftarrow 0$  to  $n$ 
2.    $V[i, 0] \leftarrow 0$ 
3. end for
4. for  $j \leftarrow 0$  to  $C$ 
5.    $V[0, j] \leftarrow 0$ 
6. end for
7. for  $i \leftarrow 1$  to  $n$  //前 $i$ 个物品
8.   for  $j \leftarrow 1$  to  $C$  //承重量 $C$ 与物品重量 $w_i$ 均为整数，故 $j$ 为整数
9.      $V[i, j] \leftarrow V[i-1, j]$ 
10.    if  $w_i \leq j$  then  $V[i, j] \leftarrow \max\{V[i, j], V[i-1, j-w_i] + v_i\}$ 
11.    end if
12.  end for
13. end for
14. return  $V[n, C]$ 
```

$$T(n) = \Theta(nC)$$



因为 $j=2 < w_2=3$; 所以 $V[2,2]=V[1,2]=3$

一个实例

背包的承重量为 $C=9$; 给定4个物品, 重量(w)分别为2, 3, 4, 5; 价值(v)依次为3, 4, 5, 7。问: 背包中最多能装的物品的总价值是对少?

	$c=0$	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7	7	7
3	0	0	3	4	4	7	8	9	9	12
4	0	0	3	4	5	7	8	10	11	12

因为 $w_2=3 \leq j=3$;

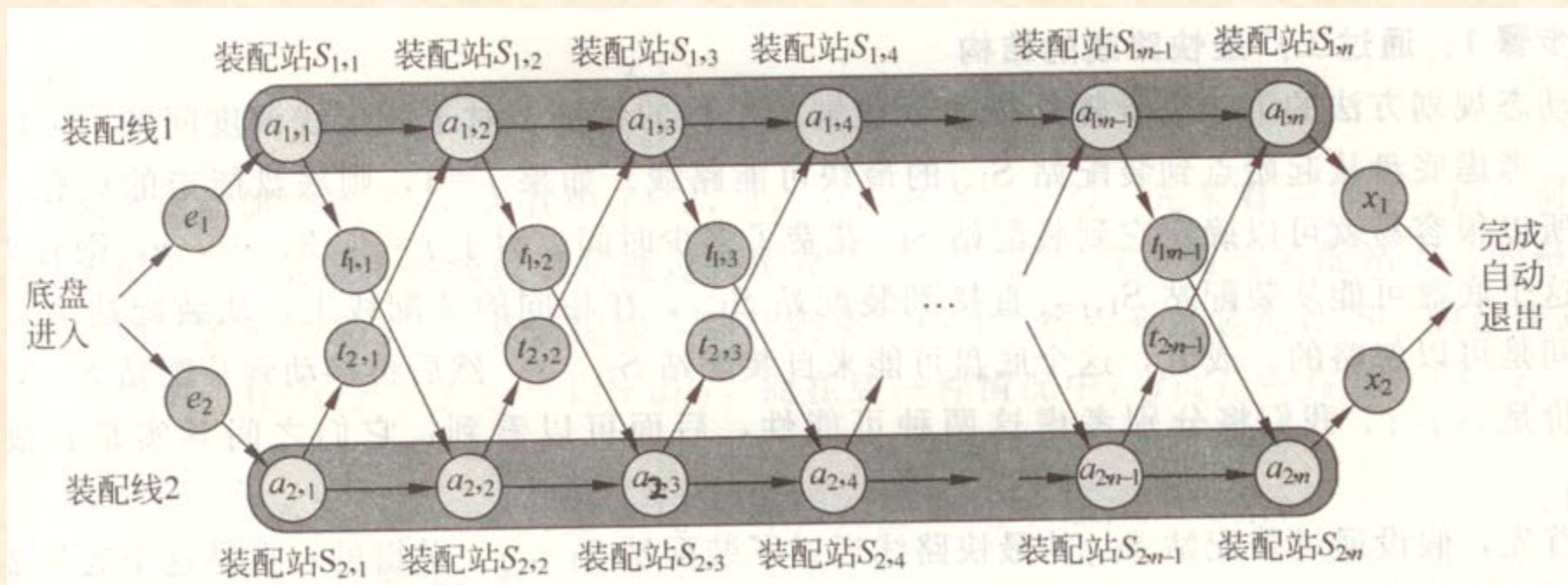
所以 $V[2, 3] = \max\{V[2-1,3], V[2-1,3-w_2]+v_2\} = \max\{V[1,3], V[1,0]+4\} = \max\{3,0+4\} = 4$

因为 $w_2=3 \leq j=5$;

所以 $V[2, 5] = \max\{V[2-1,5], V[2-1,5-w_2]+v_2\} = \max\{V[1,5], V[1,2]+4\} = \max\{3,3+4\} = 7$



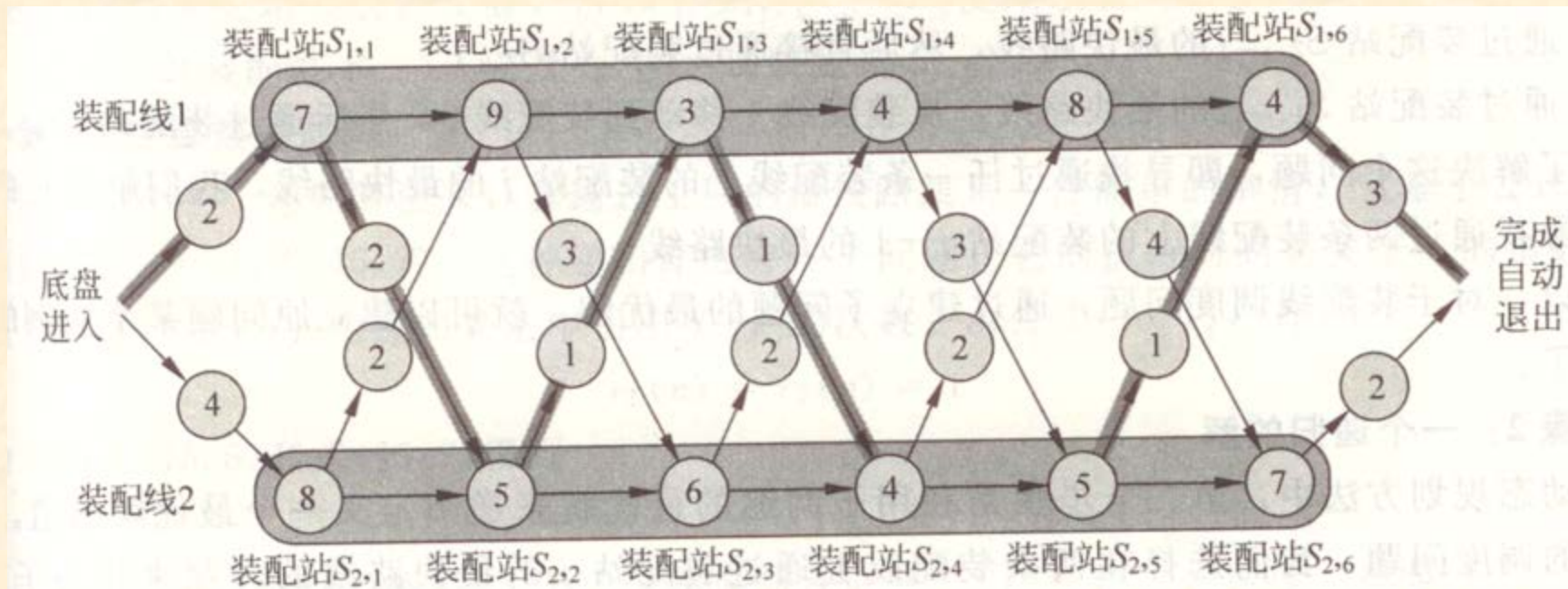
汽车装配线调度问题



- ◆某汽车制造厂家有2条装配线，分别建于不同时期，效率不同。
 - ◆汽车底盘进入生产线，要依次装配n个部件才能离开流水线。
 - ◆装配站 S_{ij} 可装配第j个部件。装配时间耗费是 a_{ij} , $i=1,2, j=1,\dots,n$.
 - ◆不同装配线之间物件转移的时间代价是 t_{ij} , $i=1,2, j=1,\dots,n$.
 - ◆同一条装配线的不同装配站之间物件转移时间可忽略。
 - ◆汽车底盘进入流水线（上线）时间耗费 e_i , 下线时间耗费 x_i , $i=1,2$.
- 如何调度，才能使总时间耗费最少？



一个例子的解

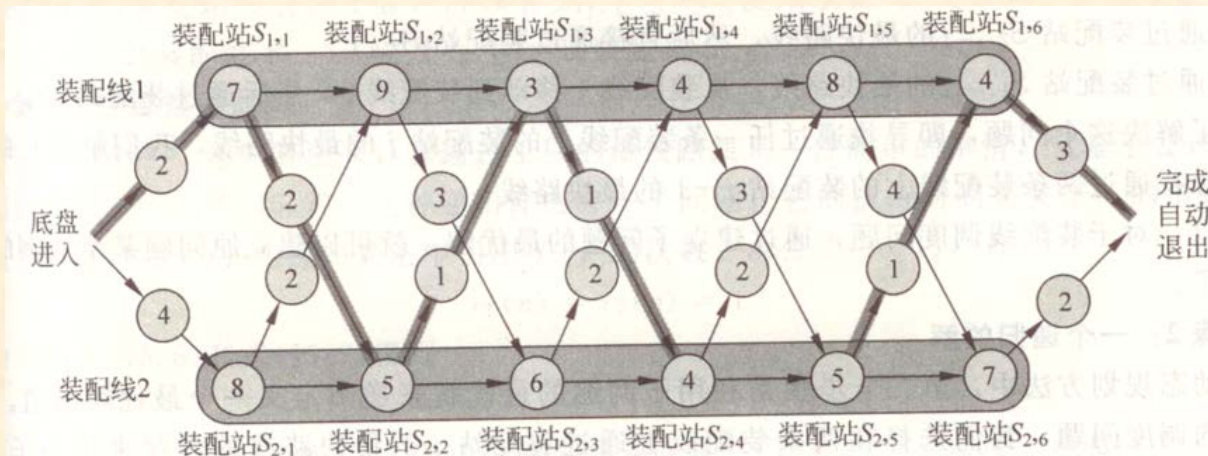




问题分析：最优解的结构

- 假设通过装配站 $S_{1,j}$ 的最快路线通过的上一站是 $S_{1,j-1}$ ，那么它必定是利用最快路线从起点到装配站 $S_{1,j-1}$ 的。（为什么？）
- 假设通过装配站 $S_{1,j}$ 的最快路线通过的上一站是 $S_{2,j-1}$ ，那么它必定是利用最快路线从起点到装配站 $S_{2,j-1}$ 的。（为什么？）

最优子结构性质：问题的最优解（通过装配站 S_{ij} 的最快路线）包含子问题（通过装配站 $S_{1,j-1}$ 或 $S_{2,j-1}$ 的最快路线）的最优解。





递归定义解的结构

通过装配站 $S_{1,j}$ 的最快路线只能是如下二者之一：

- ◆ 通过装配站 $S_{1,j-1}$ 的最快路线，然后直接进入 $S_{1,j}$
- ◆ 通过装配站 $S_{2,j-1}$ 的最快路线，然后从装配线2转移到1，时间花费是 $t_{2,j-1}$ ，然后进入装配站 $S_{1,j}$

记 $f_i[j]$ 表示一个汽车从起点到离开装配站 $S_{i,j}$ 的最快时间

$$f_1[j] = \begin{cases} e_1 + a_{1,1} & \text{if } j = 1 \\ \min(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j}) & \text{if } j \geq 2 \end{cases}$$

$$f_2[j] = \begin{cases} e_2 + a_{2,1} & \text{if } j = 1 \\ \min(f_2[j-1] + a_{2,j}, f_1[j-1] + t_{1,j-1} + a_{2,j}) & \text{if } j \geq 2 \end{cases}$$

最后，有 $f^* = \min(f_1[n] + x_1, f_2[n] + x_2)$

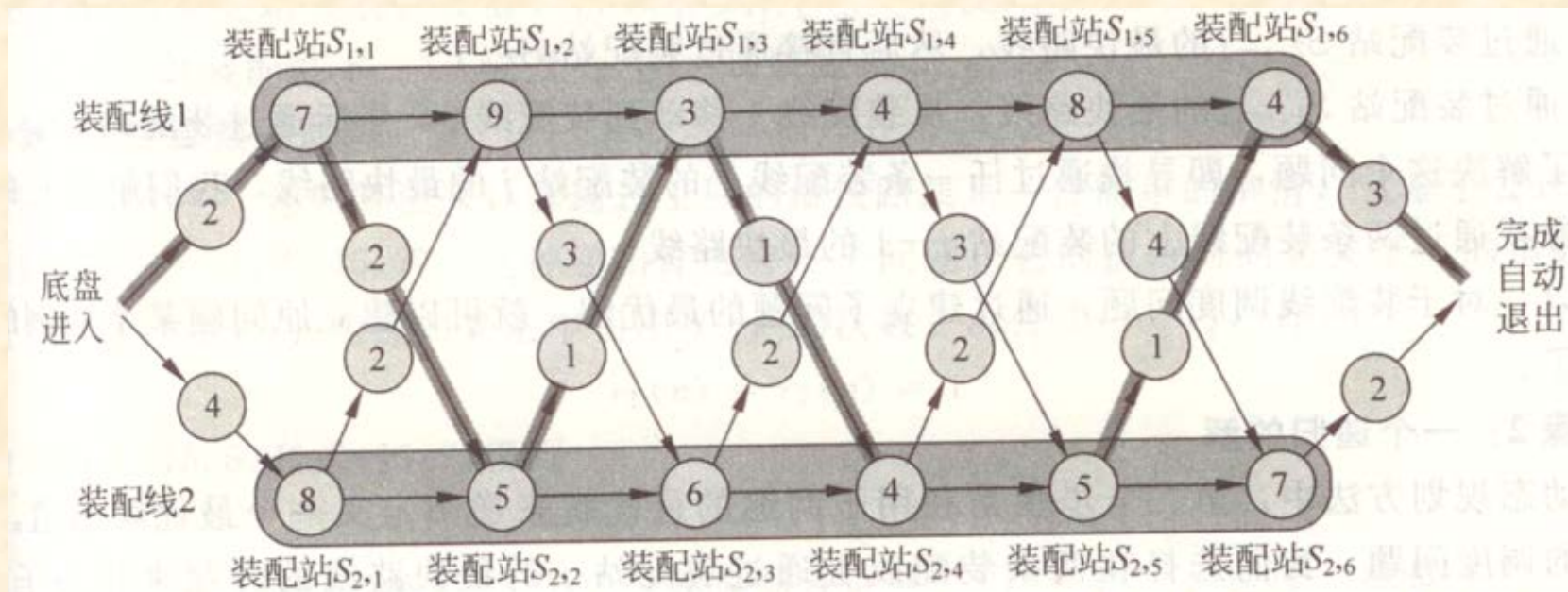


算法设计

1. $f1[1]=e1+a[1][1]$
2. $f2[1]=e2+a[2][1]$
3. For $j \leftarrow 2$ to n
4. if $f1[j-1]+a[1][j] \leq f2[j-1]+t[2][j-1]+a[1][j]$
5. then $f1[j] \leftarrow f1[j-1]+a[1][j]$; $line1[j] \leftarrow 1$
6. else $f1[j] \leftarrow f2[j-1]+t[2][j-1]+a[1][j]$; $line1[j] \leftarrow 2$
7. if $f2[j-1]+a[2][j] \leq f1[j-1]+t[1][j-1]+a[2][j]$
8. then $f2[j] \leftarrow f2[j-1]+a[2][j]$; $line2[j] \leftarrow 2$
9. else $f2[j] \leftarrow f1[j-1]+t[1][j-1]+a[2][j]$; $line2[j] \leftarrow 1$
10. End for
11. if $f1[n]+x1 \leq f2[n]+x2$
12. then $f^* \leftarrow f1[n]+x1$; $line^* \leftarrow 1$
13. else $f^* \leftarrow f2[n]+x2$
14. end if
15. Return $f^*, line1, line^*, line2, line2^*$



一个例子解



j	1	2	3	4	5	6
$f_1[j]$	9	18	20	24	32	35
$f_2[j]$	12	16	22	25	30	37

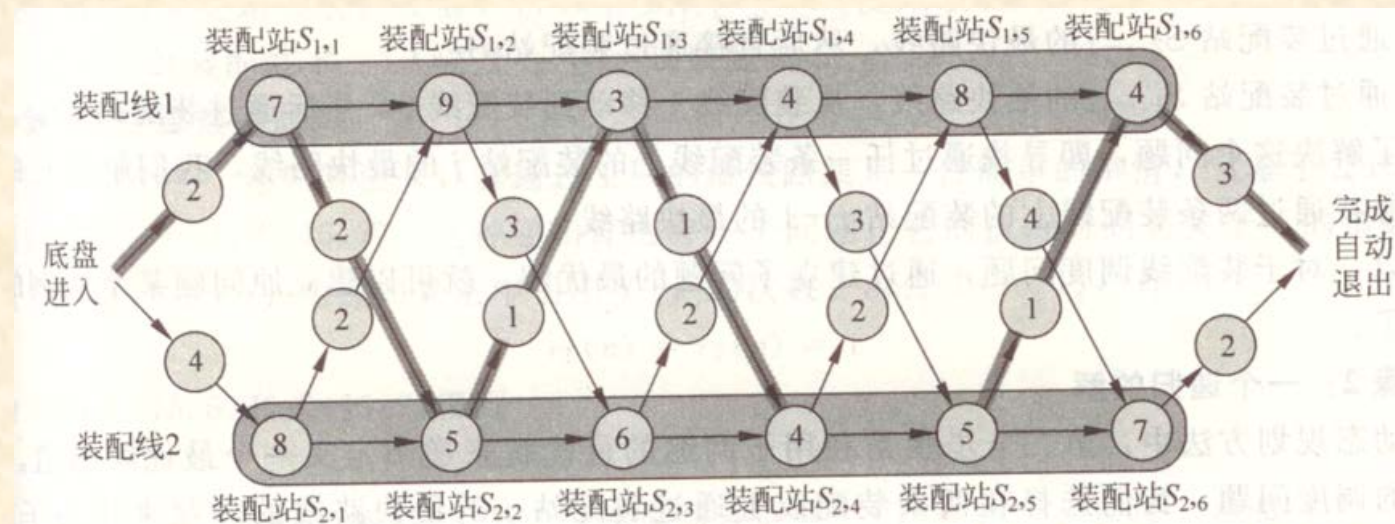
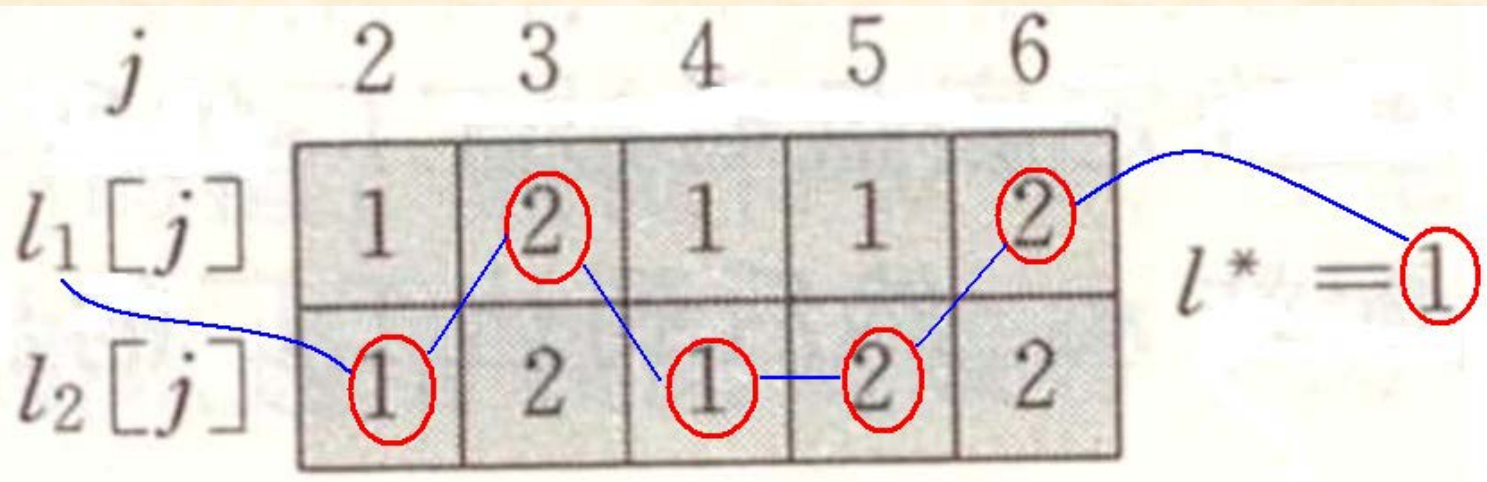
$f^* = 38$

j	2	3	4	5	6
$l_1[j]$	1	2	1	1	2
$l_2[j]$	1	2	1	2	2

$l^* = 1$



根据求解过程中的示踪信息，构造最优解





最优二叉查找树

我们准备设计一个软件，能够将英语自动翻译成法语。对于英文文章中出现的每个单词，需要查看与其等价的法语。执行这样的操作就是建立一棵二叉查找树。

显然查找的时间代价与查找长度有关。我们将查找时间最少的二叉树称之为最优二叉查找树。

插值查找是根据给定值 key 来确定进行比较的关键字 ST.elem[j], key 的查找方法。

累计发行超100万册

如果只考虑查找成功的情况，则使查找性能达最佳的判定树是其带权内路径长度之和 PH 值^①

$$PH = \sum_{i=1}^n w_i h_i \quad (9-7)$$

取最小值的二叉树。其中： n 为二叉树上结点的个数（即有序表的长度）； h_i 为第 i 个结点在二叉树上的层数；结点的权 $w_i = c p_i$ ($i=1, 2, \dots, n$)，其中 p_i 为结点的查找概率， c 为某个常量。称 PH 值取最小的二叉树为静态最优查找树 (Static Optimal Search Tree)。由于构造静态最优查找树花费的时间代价较高，因此在本书中不作详细讨论，有兴趣的读者可查阅参考书目[1]。在此向读者介绍一种构造近似最优查找树的有效算法。



最优二叉查找树

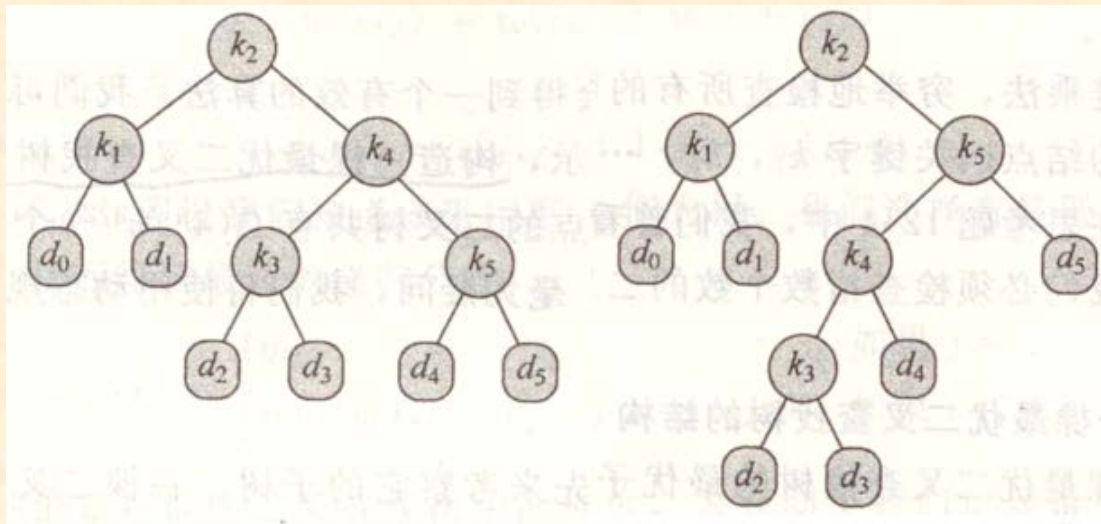
形式化描述：给定 n 个互异的关键字序列

$K=\langle k_1, k_2, \dots, k_n \rangle$ ，且关键字有序，即 $k_1 < k_2 < \dots < k_n$ ，需要构造一颗二叉查找树。每个关键字 k_i 的查找概率为 p_i 。某些搜索值不在 K 内（查找失败），可定义为 $n+1$ 个“虚拟键” $d_0, d_1, \dots, d_n$ 。其中 d_i 表示所有位于 k_i 与 k_{i+1} 之间的值，并设其被查找的概率是 q_i 。显然有：

$$\sum_{i=1}^n p_i + \sum_{i=0}^n q_i = 1$$



一个例子



i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

假设一次搜索的实际代价为检查的结点个数（即结点的深度加1，树根深度定义为0），则期望代价为

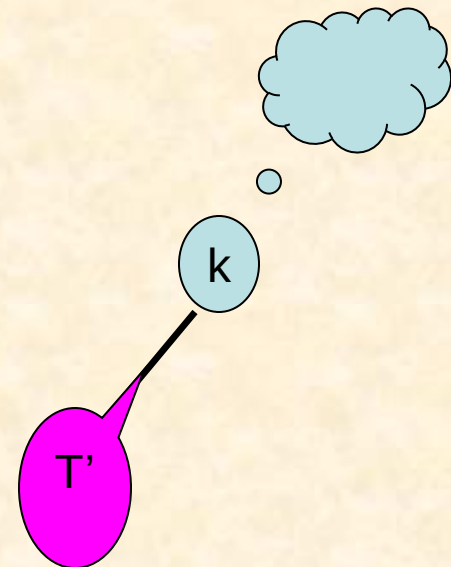
$$\begin{aligned}
 E[\text{cost}] &= \sum_{i=1}^n (\text{depth}(k_i) + 1) \cdot p_i + \sum_{i=0}^n (\text{depth}(d_i) + 1) \cdot q_i \\
 &= 1 + \sum_{i=1}^n \text{depth}(k_i) \cdot p_i + \sum_{i=0}^n \text{depth}(d_i) \cdot q_i
 \end{aligned}$$

左边的树搜索代价为2.80，右边为2.75，是最优搜索树。这个例子也说明：最优搜索树不一定是整体高度最小的树。注意：中序遍历树是关键字递增序列。



最优二叉查找树的最优子结构性质

- 最优二叉查找树 T^* 的任一子树 T' 也必定是最优的。

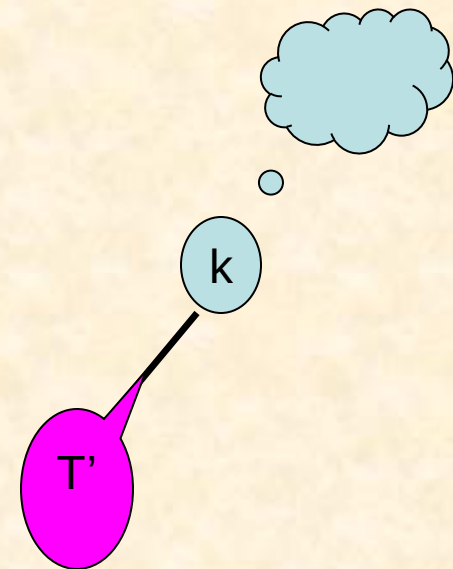


$$\begin{aligned}\text{cost}(T^*) &= C + \sum_{k_i \in T'} (\text{depth}(k_i) + 1) \cdot p_i \\ &= C + \sum_{k_i \in T'} [\text{depth}(k) + 1] \cdot p_i + \sum_{k_i \in T'} [\text{depth}(k_i) - \text{depth}(k)] \cdot p_i \\ &= C + [\text{depth}(k) + 1] \cdot \sum_{k_i \in T'} p_i \\ &\quad + \sum_{k_i \in T'} \{\text{depth}(k_i) - [\text{depth}(k) + 1] + 1\} \cdot p_i \\ &= C + C' + \text{cost}(T')\end{aligned}$$



最优二叉查找树的最优子结构性质

假如 T' 不是最优的, 存在 $\text{cost}(T'') < \text{cost}(T')$ 。
将 T' 剪贴, 而用 T'' (结点不变) 取代之, 构成树 T^{**} , 则有



$$\begin{aligned}\text{cost}(T^{**}) &= C + \sum_{k_i \in T''} (\text{depth}(k_i) + 1) \cdot p_i \\ &= C + \sum_{k_i \in T''} [\text{depth}(k) + 1] \cdot p_i + \sum_{k_i \in T''} [\text{depth}(k_i) - \text{depth}(k)] \cdot p_i \\ &= C + [\text{depth}(k) + 1] \cdot \sum_{k_i \in T''} p_i \\ &\quad + \sum_{k_i \in T''} \{\text{depth}(k_i) - [\text{depth}(k) + 1] + 1\} \cdot p_i \\ &= C + C' + \text{cost}(T'') \\ &< C + C' + \text{cost}(T') \\ &= \text{cost}(T^*)\end{aligned}$$

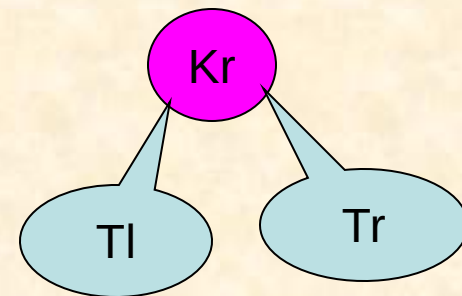
这与 T^* 已经是最优的相矛盾。



递归定义解的结构

- 考虑任一棵子树，包含了关键字 $k_i, \dots, k_j$ ，只要我们检查所有的树根 k_r ， $i \leq r \leq j$ ，并且确定包含关键字 $k_i, \dots, k_{r-1}$ 和 $k_{r+1}, \dots, k_j$ 的子树是最优的，就保证找到一棵最优的二叉查找树。代价记为 $e[i, j]$ ，有

$$e[i, j] = p_r \cdot 1 + \sum_{l=i}^{r-1} p_l \cdot [\text{depth}(k_l) + 1] + \sum_{l=i-1}^{r-1} q_l \cdot [\text{depth}(d_l) + 1] \\ + \sum_{l=r+1}^j p_l \cdot [\text{depth}(k_l) + 1] + \sum_{l=r}^j q_l \cdot [\text{depth}(d_l) + 1]$$



$$= \left\{ \sum_{l=i}^j p_l + \sum_{l=i-1}^j q_l \right\} + \left\{ \sum_{l=i}^{r-1} p_l \cdot [\text{depth}(k_l) - 1 + 1] + \sum_{l=i-1}^{r-1} q_l \cdot [\text{depth}(d_l) - 1 + 1] \right\} \\ + \left\{ \sum_{l=r+1}^j p_l \cdot [\text{depth}(k_l) - 1 + 1] + \sum_{l=r}^j q_l \cdot [\text{depth}(d_l) - 1 + 1] \right\} \\ = w[i, j] + e[i, r-1] + e[r+1, j]$$



递归定义解的结构

最优的 $e[i, j]$ 显然应该是：

$$e[i, j] = \begin{cases} \min_{i \leq r \leq j} \{e[i, r-1] + e[r+1, j] + w[i, j]\} & \text{if } i \leq j \\ q_{i-1} & \text{if } j = i-1 \end{cases}$$

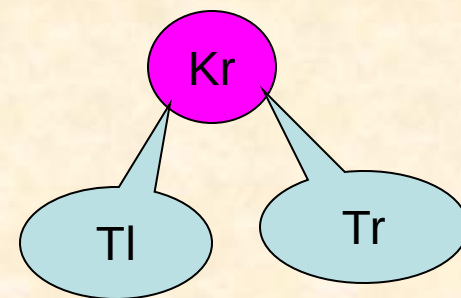
其中 $w[i, j] = \sum_{l=i}^j p_l + \sum_{l=i-1}^j q_l$

$$= \sum_{l=i}^{r-1} p_l + p_r + \sum_{l=r+1}^j p_l + \sum_{l=i-1}^{r-1} q_l + \sum_{l=r}^j q_l$$

$$= \left(\sum_{l=i}^{r-1} p_l + \sum_{l=i-1}^{r-1} q_l \right) + p_r + \left(\sum_{l=r+1}^j p_l + \sum_{l=r}^j q_l \right)$$

$$= w[i, r-1] + p_r + w[r+1, j]$$

$$= w[i, j-1] + p_j + q_j$$





最优二叉查找树算法设计

Optimal-BST(p,q,n)

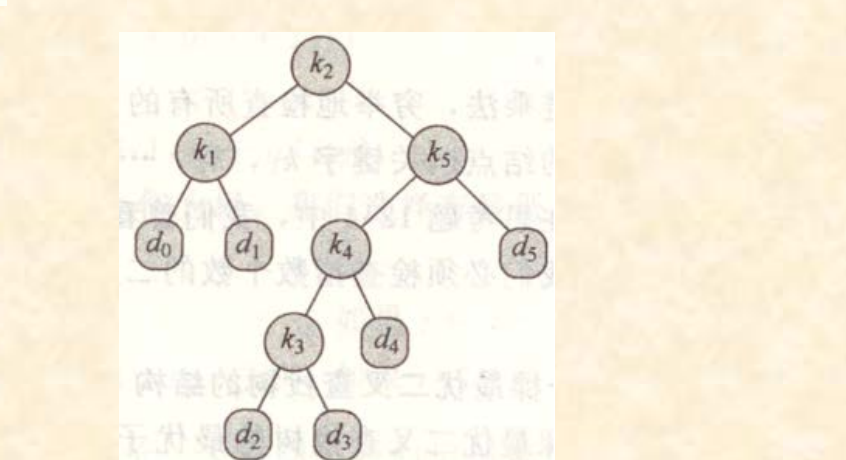
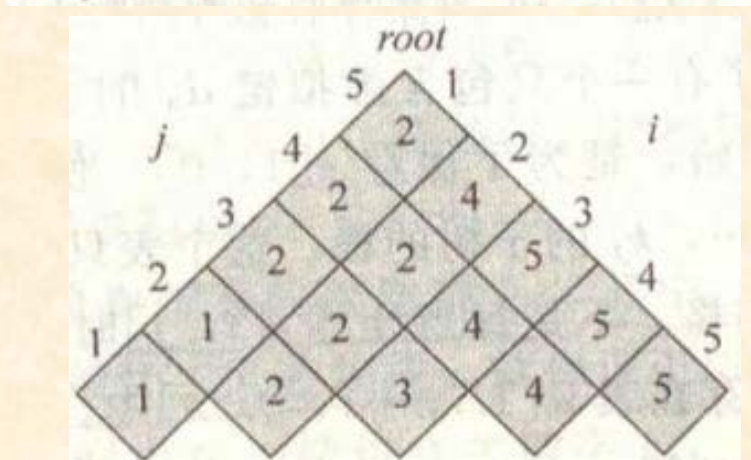
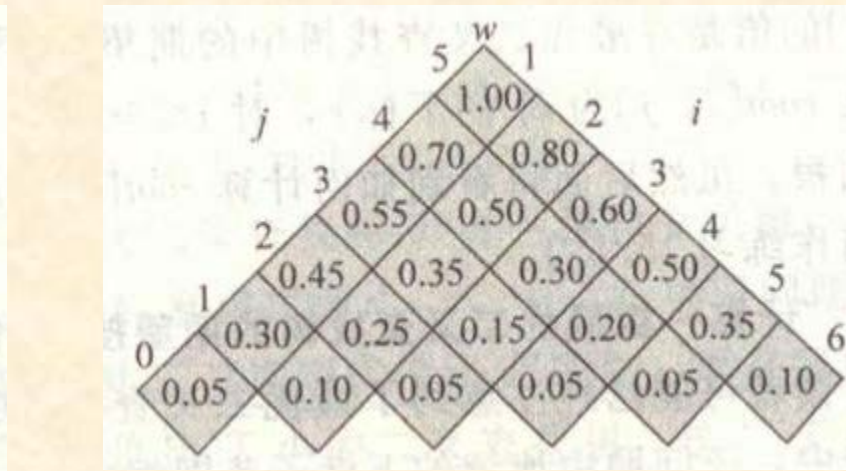
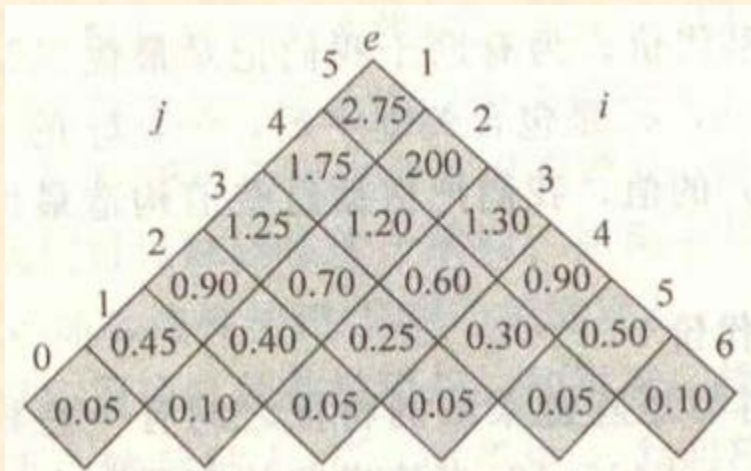
1. For $i \leftarrow 1$ to $n+1$
2. $e[i,i-1] \leftarrow q_{i-1}$; $w[i,i-1] \leftarrow q_{i-1}$
3. End for
4. For $d \leftarrow 1$ to n
5. For $i \leftarrow 1$ to $n-d+1$
6. $j \leftarrow i+d-1$
7. $e[i,j] \leftarrow \infty$
8. $w[i,j] \leftarrow w[i,j-1] + p_j + q_j$
9. For $r \leftarrow i$ to j
10. $t \leftarrow e[i,r-1] + e[r+1,j] + w[i,j]$
11. If $t < e[i,j]$
12. then $e[i,j] \leftarrow t$
13. $root[i,j] \leftarrow r$
14. End for
15. End for
16. End for
17. Return e and $root$

$$\begin{aligned}
 \sum_{d=1}^n \sum_{i=1}^{n-d+1} \sum_{r=i}^j c &= \sum_{d=1}^n \sum_{i=1}^{n-d+1} \sum_{r=i}^{i+d-1} c \\
 &= \sum_{l=1}^n \sum_{i=1}^{n-d+1} \sum_{r=1}^d c \\
 &= c \sum_{d=1}^n \sum_{i=1}^{n-d+1} d \\
 &= c \sum_{d=1}^n d(n-d+1) \\
 &= c \left(\sum_{d=1}^n (n+1)d - \sum_{l=1}^n d^2 \right) \\
 &= c \left(\frac{n(n+1)^2}{2} - \frac{n(n+1)(2n+1)}{6} \right) \\
 \text{复杂度} &= \Theta(n^3)
 \end{aligned}$$



一个最优二叉查找树的例子

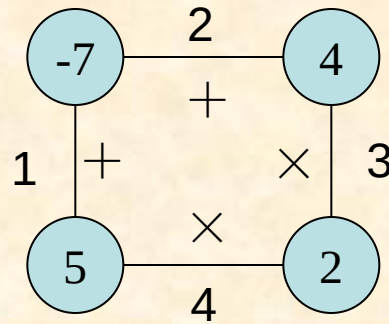
i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10





多边形游戏

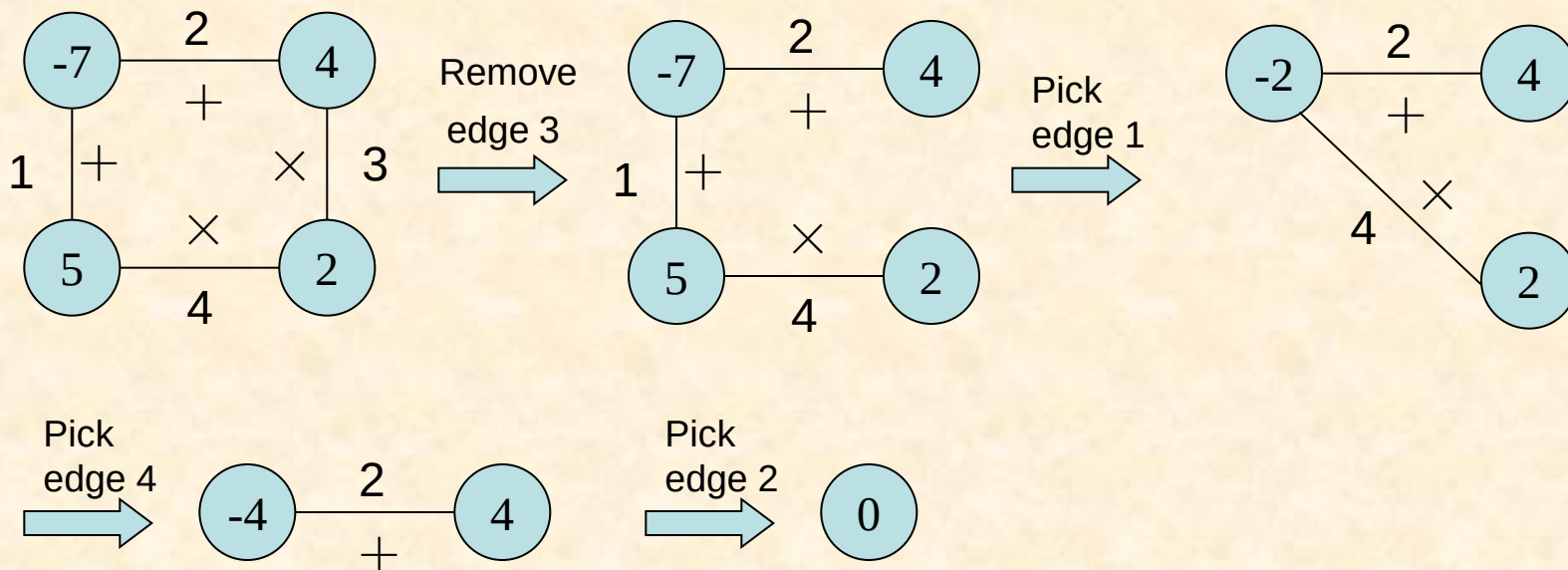
- 给定 N 个顶点的多边形，每个顶点标有一个整数，每条边上标有 $+$ (加)或是 $\times$ (乘)号，并且 N 条边按照顺时针依次编号为 $1 \sim N$ 。下图给出了一个 $N=4$ 个顶点的多边形。





游戏规则

- (1) 首先，移走一条边。
- (2) 然后进行下面的操作：
 - 选中一条边 E ，该边有两个相邻的顶点，不妨称为 V_1 和 V_2 。对 V_1 和 V_2 顶点所标的整数按照 E 上所标运算符(+或是 $\times$)进行运算，得到一个整数；用该整数标注一个新顶点，该顶点代替 V_1 和 V_2 。
 - 持续进行此操作，直到最后没有边存在，即只剩下一个顶点。该顶点的整数称为此次游戏的得分(Score)。

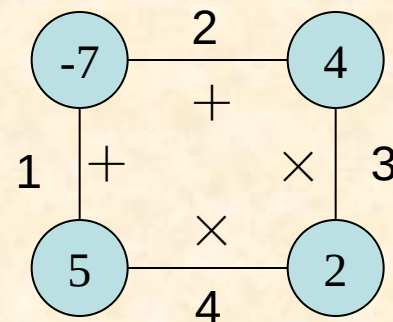




- 任务：给定一个多边形，顶点和边已按上述方式进行标注。问：按照游戏规则，最高得分(最优值)是多少？对应该最高得分，按照什么顺序移走边(最优解)？
- 输入文件：文件POLYGON.IN中存储了多边形的信息，该文件中有两行数据：
 - 第一行是一个整数N
 - 第二行按照
边 顶点 边 顶点 边 顶点
的顺序以此存放了N个顶点和N条边的标注信息。其中字符t表示+，字符x表示×。
 - 例如，下图对应的POLYGON.IN为：

4

t -7 t 4 x 2 x 5

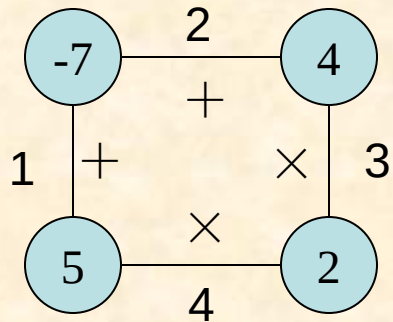




- 输出文件：文件POLYGON.OUT，该文件中有两行数据：
 - 第一行是该游戏可能的最高得分。
 - 第二行列出第一次移走哪条边(可能有多条, 如果是多条, 则按照递增顺序排列), 会导致最高得分的出现。
- 例如, 下图对应的POLYGON.OUT为:

33

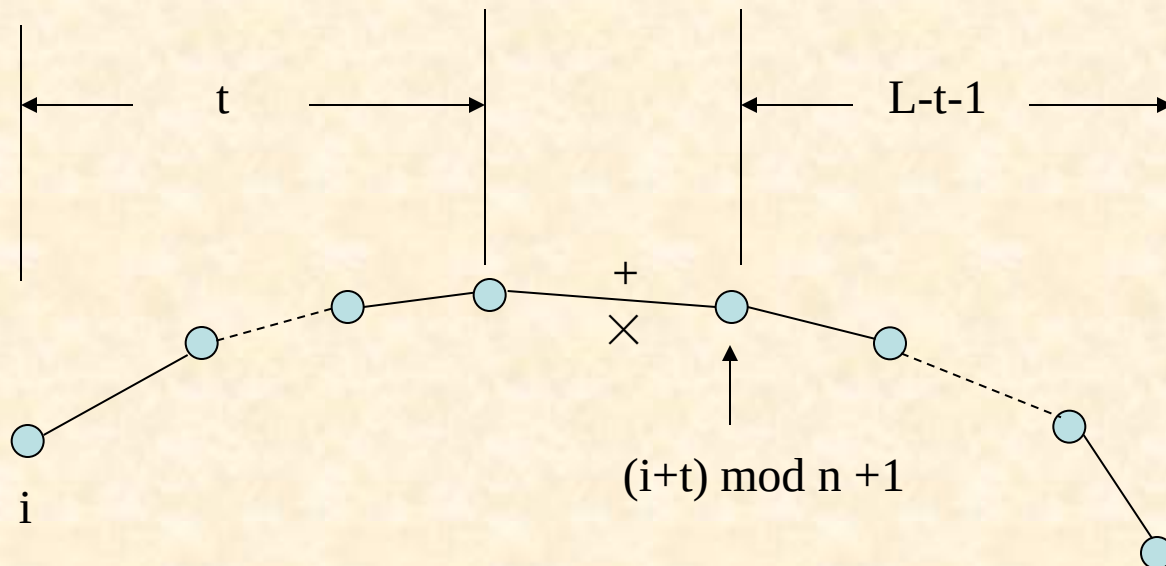
1 2





思路

- Brute-Force方法：时间复杂度为 $T(n)=\Theta(n!)$
- 分析：关键在于乘法运算同号得正，异号得负，如果a和b都是负数， $a*b$ 有可能得到一个很大的正数。因此需要同时保存子问题的最大值和最小值。
- 从顶点i开始，按顺时针长度为L的链的计算结果最小值为 $F_{\min}(i,L)$ ，最大值为 $F_{\max}(i,L)$ ，联结第i个顶点和其顺时针方向的下一个顶点 $(i \bmod n + 1)$ 的边上的运算符记为 $\text{opr}(i)$ ，则可以得到以下递归公式：
- 注意：第i个顶点沿顺时针方向后的第t个顶点的编号为： $(i+(t-1)) \bmod n + 1$



$$F_{\min}(i, L) = \begin{cases} \min_{0 \leq t \leq L-1} \{F_{\min}(i, t) + F_{\min}((i+t) \bmod n + 1, L-t-1)\}, & \text{if } \text{opr}((i+t) \bmod n + 1) = '+' \\ \min_{0 \leq t \leq L-1} \{F_{\min}(i, t) \times F_{\min}((i+t) \bmod n + 1, L-t-1), \\ F_{\min}(i, t) \times F_{\max}((i+t) \bmod n + 1, L-t-1), \\ F_{\max}(i, t) \times F_{\min}((i+t) \bmod n + 1, L-t-1) \\ F_{\max}(i, t) \times F_{\max}((i+t) \bmod n + 1, L-t-1)\}, & \text{if } \text{opr}((i+t) \bmod n + 1) = '×' \end{cases}$$



$$F_{\max}(i, L) = \begin{cases} \max_{0 \leq t \leq L-1} \{F_{\max}(i, t) + F_{\max}((i+t) \bmod n+1, L-t-1)\}, & \text{if } \text{opr}((i+t) \bmod n+1) = '+' \\ \max_{0 \leq t \leq L-1} \{F_{\min}(i, t) \times F_{\min}((i+t) \bmod n+1, L-t-1), \\ F_{\min}(i, t) \times F_{\max}((i+t) \bmod n+1, L-t-1), \\ F_{\max}(i, t) \times F_{\min}((i+t) \bmod n+1, L-t-1) \\ F_{\max}(i, t) \times F_{\max}((i+t) \bmod n+1, L-t-1)\}, & \text{if } \text{opr}((i+t) \bmod n+1) = ' \times ' \end{cases}$$

$$F_{\min}(i, 0) = V(i)$$

$$F_{\max}(i, 0) = V(i)$$

- 公式中 $(i+t) \bmod n+1$ 是顶点 i 顺时针方向后的第 $t+1$ 顶点的编号， $V(i)$ 为顶点 i 上所标的数字， $\text{opr}(i)$ 为联结第 i 个顶点和其顺时针方向的下一个顶点 $i \bmod n+1$ 的边上的运算符。根据上述公式，我们可以求出所有的 $F_{\max}(i, n)$ ， $(i=1, 2, \dots, n)$ ，其中的最大值就是问题的解。



高性能计算机任务分配问题

问题描述:

现在有一项大型计算任务需要完成。这项大型计算任务包含 n_A 个 **A 类子任务** 和 n_B 个 **B 类子任务**；这些子任务**互不相关**，并且每个 A 类子任务的工作量都是一样的，每个 B 类子任务的工作量也是一样的(每个 A 和 B 类子任务的工作量不一定相同)。各个子任务之间没有执行顺序上的要求。

假设超级计算机拥有 p 个计算节点，每个节点都包括一个**串行处理器**、**本地主存**和**高速 cache**。然而由于常年使用和不连贯的升级，各个计算节点的**计算能力并不对称**。

一个节点的**计算能力**包括如下几个方面：每个节点都有三种工作状态：**待机状态**、**A 类状态**和**B 类状态**。其中，执行 A 类任务时为 A 类状态；执行 B 类任务时为 B 类状态；待机状态下不执行计算。所有的处理器在开始工作之前都处于待机状态，而从其它的状态转入 A 类状态或 B 类状态(包括 A 类状态和 B 类状态之间相互转换)，都要花费一定的启动时间，并且对于不同的处理节点，这个状态转换启动时间不一定相同。用两个正整数 t_i^A 和 t_i^B ($i=1,2,\dots,p$) 分别表示计算节点 i 转入 A 类状态和 B 类状态所需的启动时间(单位：ns)。



一个节点在连续处理同一类任务的时候，执行时间(不含状态转换的时间)随任务量(这一类子任务的数目)的平方增长，即：

若节点 i 连续处理 x 个 A 类子任务，则对应的执行时间为： $t=k_i^A \cdot x^2$ ；

类似地，若节点 i 连续处理 x 个 B 类子任务，对应的执行时间为： $t=k_i^B \cdot x^2$ 。

其中， k_i^A 和 k_i^B 是系数，单位是 ns， $i=1,2,\dots,p$ 。

任务分配必须在所有计算开始之前完成。所谓任务分配，即给每个计算节点设置一个任务队列，队列由一串 A 类和 B 类子任务组成，两类子任务可以交错排列。

计算开始后，各计算节点分别从各自的子任务队列中顺序读取计算任务并执行，队列中连续的同类子任务将由该计算节点一次性读出，队列中一串连续的同类子任务不能被分成两部分执行。



编程任务:

现在需要你编写程序, 给这 p 个节点安排计算任务, 使得这个工程计算任务能够尽早完成。假定任务安排好后不再变动, 而且所有的节点都同时开始运行, 任务安排的目标是使最后结束计算的节点的完成时间尽可能早。

输入输出:

输入文件名是 hpc.in。

文件的第一行是对计算任务的描述, 包括两个正整数 n_A 和 n_B , 分别是 A 类和 B 类子任务的数目, 两个整数之间由一个空格隔开。

文件的后面部分是对此计算机的描述:

文件第二行是一个整数 p , 即计算节点的数目。

随后连续的 p 行按顺序分别描述各个节点的信息, 第 i 个节点由第 $i+2$ 行描述, 该行包括下述四个正整数 (相邻两个整数之间有一个空格): t_i^A t_i^B k_i^A k_i^B

输出文件:

输出文件名是 hpc.out。其中只有一行, 包含有一个正整数, 即从各节点开始计算到任务完成所用的时间。

样例:

设输入文件 hpc.in 为

```
5 5
3
15 10 6 4
70 100 7 2
30 70 1 6
```

对应的输出文件 hpc.out 为

数据说明:

$$1 \leq n_A \leq 60, 1 \leq n_B \leq 60$$

$$1 \leq p \leq 20$$

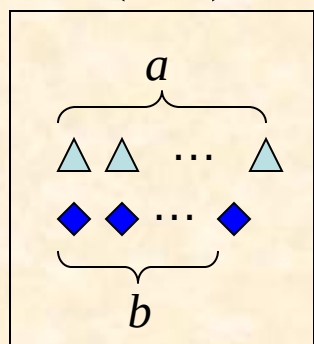
$$1 \leq t_A \leq 1000, 1 \leq t_B \leq 1000, 1 \leq k_A \leq 50, 1 \leq k_B \leq 50$$



△ - 1份A类子任务

◆ - 1份B类子任务

任务 (a, b)



计算节点

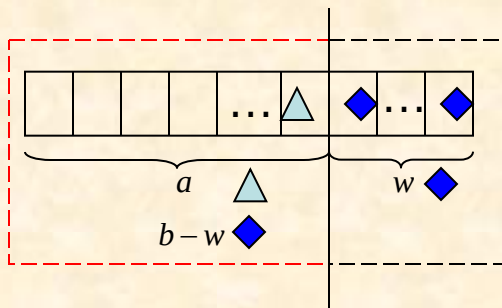


$f_i(a, b) \rightarrow ?$

第i个节点所需最短计算时间:

$p=1$

$f_1(n_A, n_B)$



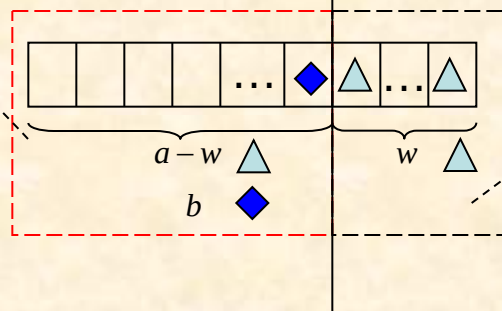
分析:节点i完成所分配的任务时,最后所完成的子任务不是A类子任务就是B类子任务.

$P_i(a, b, 0)$ — 节点i完成任务(a,b)所花的最短时间, (最后完成的一项子任务是A类子任务)

$P_i(a, b, 1)$ — 节点i完成任务(a,b)所花的最短时间, (最后完成的一项子任务是B类子任务)

$$P_i(a, b, 0) = \min_{1 \leq w \leq a} \{P_i(a-w, b, 1) + t_i^A + k_i^A \cdot w^2\}$$

$$P_i(a, b, 1) = \min_{1 \leq w \leq b} \{P_i(a, b-w, 0) + t_i^B + k_i^B \cdot w^2\}$$



$$f_i(a, b) = \min\{P_i(a, b, 0), P_i(a, b, 1)\}$$



边界值:

$$\begin{cases} P_i(a, b, 0) = 0 \\ P_i(a, b, 1) = 0 \end{cases}, a = 0 \wedge b = 0$$

$$\begin{cases} P_i(a, b, 0) = \infty \\ P_i(a, b, 1) = t_i^B + k_i^B \cdot b^2 \end{cases}, a = 0 \wedge b \neq 0$$

$$\begin{cases} P_i(a, b, 0) = t_i^A + k_i^A \cdot a^2 \\ P_i(a, b, 1) = \infty \end{cases}, a \neq 0 \wedge b = 0$$

∞ 表示无穷大, 表示此情形是不可能发生的。

现在我们已经知道节点*i*完成任务(a,b)所需要的最短时间为 $f_i(a,b)$, 下面的问题就变成, 有 (n_A, n_B) 项任务, 有*p*个节点可以并行地完成该任务, 每个节点完成任务(a,b)所需要的最短时间为 $f_i(a,b)$ 已知, 如何分配任务使得完成全部任务的时间最短。这样这个问题已经变成一个很简单且经典的动态规划问题了。

设 $C_i(a,b)$ 表示将任务(a,b)分配给前*i*个节点, 并行完成这些任务所需要的最短的时间。则原来题目所求就是 $C_p(n_A, n_B)$ 。我们可以写出递归式:

$$C_i(a, b) = \min\{\max\{C_{i-1}(a - w_1, b - w_2), f_i(w_1, w_2)\}\}$$

其中: $0 \leq w_1 \leq a, 0 \leq w_2 \leq b, (w_1 \neq 0 \vee w_2 \neq 0) \wedge (w_1 \neq a \vee w_2 \neq b)$

解释如下...



$$C_i(a, b) = \min\{\max\{C_{i-1}(a - w_1, b - w_2), f_i(w_1, w_2)\}\}$$

解释: 如果从1到i的节点(前i个节点)要完成的任务总数是(a,b)的话,
我们可以分配给第i个节点(w_1, w_2)的任务,
它所需的最短时间为 $f_i(w_1, w_2)$ (这在前面已经求出来了);
而前1到i-1个节点总共要完成任务($a-w_1, b-w_2$),
所需最短时间为 $C_{i-1}(a-w_1, b-w_2)$ 。

因为是所有的节点是并行地完成任务

(取决于最后一个完成时间, 反水桶原理),

所以从1到i的节点完成任务(a,b)所需的最短时间就是:

$$\max\{C_{i-1}(a-w_1, b-w_2), f_i(w_1, w_2)\}。$$

$$0 \leq w_1 \leq a, 0 \leq w_2 \leq b, (w_1 \neq 0 \vee w_2 \neq 0) \wedge (w_1 \neq a \vee w_2 \neq b)$$

解释: w_1 可以取0到a, w_2 可以取0到b, 但是 w_1, w_2 不能同时取上界a,b, 也不能同时取下界0.

实现方法: 编程、或手动填写表格



本章小结

动态规划算法的基本要素

- (1) 最优子结构性质
- (2) 重叠子问题性质

设计动态规划算法的步骤。

- (1) 找出最优解的性质，并刻画其结构特征。
- (2) 递归地定义最优值。
- (3) 以自底向上的方式计算出最优值。
- (4) 根据计算最优值时得到的信息，构造最优解。



作业

- A: 7.3, 7.5, 7.9